

BIBLE ASYNCIO – RÉFÉRENCE EXHAUSTIVE

LÉGENDE : [loop] = event loop | [coro] = coroutine | [task] = Task
[sync] = synchronisation | [io] = I/O | [py] = Python

[coro] Bases – async / await

DESCRIPTION : Déclare et exécute des coroutines asynchrones
POURQUOI : Concurrency sans threads pour I/O-bound (réseau, fichiers, BDD)
CONTEXTE : Serveurs, clients HTTP, scraping, pipelines asynchrones

DÉCLARATION :

```
async def ma_coroutine():          Coroutine (retourne un coroutine object)
    return 42
```

```
async def avec_attente():
    résultat = await autre_coroutine()  Suspendre et attendre
    return résultat
```

```
async def exemple():
    await asyncio.sleep(1)             Attendre 1 seconde (non-bloquant)
    valeur = await fetch_data()
    return valeur
```

POINT D'ENTRÉE :

```
import asyncio
```

```
asyncio.run(ma_coroutine())          Lance et attend (Python 3.7+)
asyncio.run(ma_coroutine(), debug=True)  Mode debug
asyncio.run(main())                   Pattern standard
```

```
# Python 3.11+
asyncio.run(main(), loop_factory=uvloop.new_event_loop)
```

RÈGLES FONDAMENTALES :

```
# Une coroutine seule ne s'exécute pas – il faut l'awaiter ou en faire une Task
coro = ma_coroutine()                Objet coroutine (non exécuté)
await coro                           Exécution (depuis une coroutine)
asyncio.run(coro)                     Exécution (depuis code synchrone)
```

```
# await est uniquement valide dans une fonction async
# Les fonctions bloquantes (time.sleep, requests.get...) BLOQUENT la boucle
# Utiliser asyncio.sleep() au lieu de time.sleep()
# Utiliser aiohttp au lieu de requests
# Utiliser asyncpg / aiomysql au lieu de psycopg2 / sqlite3
```

INSPECTER UNE COROUTINE :

```
import inspect
inspect.iscoroutine(obj)              True si objet coroutine
inspect.iscoroutinefunction(fn)       True si fonction async def
inspect.isawaitable(obj)              True si awaitable (coro, Task, Future)
asyncio.iscoroutine(obj)
asyncio.iscoroutinefunction(fn)
asyncio.isfuture(obj)
```

[task] Tasks – asyncio.Task

DESCRIPTION : Planifie une coroutine pour exécution concurrente
POURQUOI : Exécuter plusieurs coroutines en parallèle
CONTEXTE : Requêtes HTTP concurrentes, pipelines parallèles

CRÉATION :

```
task = asyncio.create_task(ma_coroutine())  Planifier (Python 3.7+)
task = asyncio.create_task(coro(), name="ma_tâche")  Avec nom
task = asyncio.ensure_future(coro())          Planifier (ancienne API)
task = loop.create_task(coro())               Via la boucle
```

AWAIT :

```
résultat = await task                Attendre la fin + récupérer résultat
await asyncio.wait_for(task, timeout=5.0)  Attendre avec timeout
```

ANNULATION :

```

task.cancel()
task.cancel(msg="raison")
await task
task.cancelled()
asyncio.current_task()

# Annulation propre
async def ma_tache():
    try:
        await asyncio.sleep(10)
    except asyncio.CancelledError:
        # Nettoyage
        raise

```

Demander l'annulation
Avec message (Python 3.9+)
Lève CanceledError si annulée
True si annulée
Task courante

Toujours re-raise CanceledError

ÉTAT :

```

task.done()
task.result()
task.exception()
task.cancelled()
task.get_name()
task.set_name("nouveau_nom")
task.get_coro()

```

True si terminée (succès, erreur, annulée)
Résultat (lève si pas done ou erreur)
Exception levée (None si succès)
True si annulée
Nom de la task
Coroutine associée

CALLBACKS :

```

task.add_done_callback(fn)
task.remove_done_callback(fn)

```

Appeler fn(task) quand terminée
Supprimer le callback

GROUPES DE TÂCHES (Python 3.11+) :

```

async with asyncio.TaskGroup() as tg:
    t1 = tg.create_task(coro1())
    t2 = tg.create_task(coro2())
    t3 = tg.create_task(coro3(), name="t3")
# Attend toutes les tâches – annule toutes si l'une lève une exception
# Les résultats sont disponibles via t1.result(), t2.result()...

# Gestion d'erreurs multiples
try:
    async with asyncio.TaskGroup() as tg:
        tg.create_task(peut_échouer())
        tg.create_task(peut_échouer())
except* ValueError as eg:
    for exc in eg.exceptions:
        print(exc)
except* IOError as eg:
    pass

```

ExceptionGroup (Python 3.11+)

[task] Exécution concurrente – gather, wait, as_completed

DESCRIPTION : Lance plusieurs coroutines en parallèle et attend les résultats
POURQUOI : Maximiser l'utilisation de la boucle, réduire la latence totale
CONTEXTE : Requêtes HTTP batch, lectures fichiers multiples

GATHER :

```

résultats = await asyncio.gather(coro1(), coro2(), coro3())
# Retourne liste [r1, r2, r3] dans l'ordre des arguments
résultats = await asyncio.gather(*[coro(i) for i in range(10)])
résultats = await asyncio.gather(coro1(), coro2(),
                                return_exceptions=True)
# Avec return_exceptions=True : exceptions dans la liste au lieu de lever

# Annuler toutes si l'une échoue (comportement par défaut de gather)
# Sans return_exceptions : la première exception annule les autres et propage

```

Ne pas lever les exceptions

WAIT :

```

tâches = [asyncio.create_task(c) for c in coroutines]
done, pending = await asyncio.wait(tâches)
done, pending = await asyncio.wait(tâches, timeout=5.0)
done, pending = await asyncio.wait(tâches,
                                return_when=asyncio.ALL_COMPLETED)
done, pending = await asyncio.wait(tâches,
                                return_when=asyncio.FIRST_COMPLETED)
done, pending = await asyncio.wait(tâches,

```

Toutes terminées (défaut)
Dès la première

```

for t in done:
    try:
        print(t.result())
    except Exception as e:
        print(f"Erreur : {e}")
for t in pending:
    t.cancel()

```

AS_COMPLETED :

```

tâches = [asyncio.create_task(c) for c in coroutines]
for coro in asyncio.as_completed(tâches):
    résultat = await coro           Résultats dans l'ordre de fin
for coro in asyncio.as_completed(tâches, timeout=10.0):
    try:
        résultat = await coro
    except asyncio.TimeoutError:
        pass

```

WAIT_FOR (timeout sur une coroutine) :

```

try:
    résultat = await asyncio.wait_for(ma_coro(), timeout=5.0)
except asyncio.TimeoutError:
    print("Timeout !")
# Annule la coroutine si timeout

# Shielding contre annulation
try:
    résultat = await asyncio.wait_for(
        asyncio.shield(ma_coro()), timeout=5.0)
except asyncio.TimeoutError:
    pass
                                ma_coro continue malgré le timeout

```

SHIELD :

```

task = asyncio.create_task(opération_critique())
try:
    await asyncio.shield(task)           La task ne sera pas annulée par cancel()
except asyncio.CancelledError:
    pass                                shield levait CancelledError mais task continue

```

TIMEOUT (Python 3.11+) :

```

try:
    async with asyncio.timeout(5.0):      Context manager timeout
        await ma_longue_opération()
except asyncio.TimeoutError:
    pass

async with asyncio.timeout(None):         Pas de timeout
    pass

# Modifier le deadline depuis l'intérieur
async with asyncio.timeout(5.0) as cm:
    cm.reschedule(asyncio.get_event_loop().time() + 10) Repousser
    await ma_coro()

```

SEMAPHORE POUR LIMITER LA CONCURRENCE :

```

sema = asyncio.Semaphore(10)             Max 10 coroutines simultanées
async def tâche_limitée(url):
    async with sema:
        return await fetch(url)
await asyncio.gather(*[tâche_limitée(u) for u in urls])

```

EXEMPLES :

```

# Fetch 100 URLs en parallèle limité à 10 simultanés
async def fetch_tous(urls, limite=10):
    sema = asyncio.Semaphore(limite)
    async def fetch_un(url):
        async with sema:
            return await fetch(url)
    return await asyncio.gather(*[fetch_un(u) for u in urls],
                                return_exceptions=True)

```

DESCRIPTION : Gère l'exécution des coroutines, I/O, timers et callbacks
 POURQUOI : Comprendre le cycle de vie pour des cas avancés
 CONTEXTE : Intégration avec code synchrone, tests, debugging

ACCÈS :

loop = asyncio.get_event_loop()	Boucle courante (ou en crée une si none)
loop = asyncio.get_running_loop()	Boucle en cours d'exécution (lève si none)
loop = asyncio.new_event_loop()	Nouvelle boucle
asyncio.set_event_loop(loop)	Définir la boucle courante

EXÉCUTION :

loop.run_until_complete(coro)	Exécuter jusqu'à fin de la coro
loop.run_until_complete(asyncio.gather(*coros))	
loop.run_forever()	Tourner jusqu'à stop()
loop.stop()	Arrêter run_forever()
loop.close()	Fermer la boucle (irréversible)
loop.is_running()	True si en cours d'exécution
loop.is_closed()	True si fermée
loop.run_until_complete(loop.shutdown_asyncgens())	Fermer les générateurs async

PLANIFICATION :

loop.call_soon(callback, *args)	Planifier un callback (prochain tour)
loop.call_soon(callback, *args, context=ctx)	
loop.call_soon_threadsafe(callback, *args)	Thread-safe (depuis autre thread)
loop.call_later(délai, callback, *args)	Planifier dans délai secondes
loop.call_at(when, callback, *args)	Planifier à l'heure absolue
loop.time()	Horloge monotonique de la boucle
loop.call_later(1.0, lambda: print("ls"))	

FUTURES :

fut = loop.create_future()	Créer un Future
fut.set_result(valeur)	Résoudre
fut.set_exception(exc)	Rejeter
fut.result()	Résultat (lève si pas done)
fut.exception()	Exception (None si succès)
fut.done()	True si terminé
fut.cancel()	Annuler
fut.cancelled()	True si annulé
fut.add_done_callback(fn)	
await fut	Attendre depuis une coroutine

EXÉCUTION DANS UN THREAD / PROCESSUS :

```
# Exécuter une fonction bloquante dans un thread pool
résultat = await loop.run_in_executor(None, fn_bloquante, arg1, arg2)
résultat = await loop.run_in_executor(executor, fn_bloquante)

from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor
with ThreadPoolExecutor(max_workers=4) as pool:
    résultat = await loop.run_in_executor(pool, fn_bloquante, arg)
with ProcessPoolExecutor(max_workers=4) as pool:
    résultat = await loop.run_in_executor(pool, fn_cpu_intensive, arg)

# Raccourci Python 3.9+
résultat = await asyncio.to_thread(fn_bloquante, arg1, arg2)
```

SIGNALS :

```
loop.add_signal_handler(signal.SIGINT, callback)
loop.add_signal_handler(signal.SIGTERM, lambda: loop.stop())
loop.remove_signal_handler(signal.SIGINT)
```

DEBUG :

loop.set_debug(True)	Activer mode debug
loop.get_debug()	Lire l'état debug
asyncio.run(main(), debug=True)	Debug via asyncio.run
# Variable d'env PYTHONASYNCIODEBUG=1	Debug global
# Mode debug : avertissement si coroutine non awaited, si exécution > 100ms	

POLITIQUE DE BOUCLE :

asyncio.get_event_loop_policy()	Politique courante
asyncio.set_event_loop_policy(policy)	Changer la politique
asyncio.DefaultEventLoopPolicy()	Défaut
# uvloop (plus rapide, Unix seulement) :	
# import uvloop	

```
# asyncio.set_event_loop_policy(uvloop.EventLoopPolicy())
# ou asyncio.run(main(), loop_factory=uvloop.new_event_loop)
```

INTROSPECTION :

<code>asyncio.all_tasks()</code>	Toutes les tâches en cours
<code>asyncio.all_tasks(loop=loop)</code>	
<code>asyncio.current_task()</code>	Tâche courante (ou None)
<code>len(asyncio.all_tasks())</code>	Nombre de tâches actives

[sync] Synchronisation – Lock, Event, Condition, Semaphore, Queue

DESCRIPTION : Primitives de synchronisation entre coroutines
 POURQUOI : Coordonner l'accès aux ressources partagées
 CONTEXTE : Ressources partagées, producteur/consommateur, barrières

— LOCK —

<code>lock = asyncio.Lock()</code>	
<code>async with lock:</code> <code> accès_exclusif()</code>	Acquérir et libérer automatiquement
<code>await lock.acquire()</code>	Acquérir manuellement
<code>lock.release()</code>	Libérer manuellement
<code>lock.locked()</code>	True si verrouillé

— RLOCK (réentrant) —

Pas de RLock natif dans asyncio – utiliser `asyncio.Lock()` avec compteur
 # Ou la lib `aiorwlock` (pip install `aiorwlock`)

— EVENT —

<code>event = asyncio.Event()</code>	
<code>await event.wait()</code>	Attendre que l'event soit set
<code>event.set()</code>	Déclencher (réveille tous les waiters)
<code>event.clear()</code>	Remettre à zéro
<code>event.is_set()</code>	True si déclenché
 # Exemple : démarrage coordonné	
<code>prêt = asyncio.Event()</code>	
<code>async def producteur():</code> <code> await initialiser()</code> <code> prêt.set()</code>	
<code>async def consommateur():</code> <code> await prêt.wait()</code> <code> await traiter()</code>	

— CONDITION —

<code>cond = asyncio.Condition()</code>	
<code>cond = asyncio.Condition(lock)</code>	Avec lock existant
<code>async with cond:</code> <code> await cond.wait()</code> <code> await cond.wait_for(lambda: prêt)</code> <code> cond.notify()</code> <code> cond.notify_all()</code> <code> cond.notify(n=3)</code>	Attendre + acquérir lock Attendre condition Réveiller un waiter Réveiller tous les waiters Réveiller n waiters

— SEMAPHORE —

<code>sema = asyncio.Semaphore(5)</code>	Compteur = 5 (5 slots)
<code>sema = asyncio.BoundedSemaphore(5)</code>	Lève si <code>release()</code> dépasse initial
<code>async with sema:</code> <code> opération()</code> <code>await sema.acquire()</code> <code>sema.release()</code> <code>sema.locked()</code>	Décrémente à l'entrée, incrémente à la sortie True si compteur = 0

— QUEUE —

<code>q = asyncio.Queue()</code>	File FIFO illimitée
<code>q = asyncio.Queue(maxsize=10)</code>	File bornée (bloque si pleine)
<code>q = asyncio.LifoQueue()</code>	File LIFO (pile)
<code>q = asyncio.LifoQueue(maxsize=10)</code>	
<code>q = asyncio.PriorityQueue()</code>	File par priorité (heapq)

```

q = asyncio.PriorityQueue(maxsize=10)

await q.put(item)           Ajouter (attend si pleine)
q.put_nowait(item)         Ajouter sans attendre (QueueFull si pleine)
item = await q.get()       Retirer (attend si vide)
item = q.get_nowait()      Retirer sans attendre (QueueEmpty si vide)
q.task_done()             Signaler fin de traitement d'un item
await q.join()             Attendre que tous les items soient traités
q.empty()                 True si vide
q.full()                  True si pleine
q.qsize()                 Nombre d'items

# Pattern producteur / consommateur
async def producteur(q):
    for item in données:
        await q.put(item)
    await q.put(None)       Sentinel de fin

async def consommateur(q):
    while True:
        item = await q.get()
        if item is None:
            break
        await traiter(item)
        q.task_done()

async def main():
    q = asyncio.Queue(maxsize=100)
    prod = asyncio.create_task(producteur(q))
    cons = asyncio.create_task(consommateur(q))
    await asyncio.gather(prod, cons)

# Pattern avec join
async def travailler(q):
    while True:
        item = await q.get()
        await traiter(item)
        q.task_done()

async def main():
    q = asyncio.Queue()
    workers = [asyncio.create_task(travailler(q)) for _ in range(5)]
    for item in données:
        await q.put(item)
    await q.join()           Attendre fin de tous les items
    for w in workers:
        w.cancel()

```

[io] Streams – asyncio.StreamReader / StreamWriter

DESCRIPTION : I/O réseau asynchrone (TCP/Unix sockets)
 POURQUOI : Serveurs et clients TCP sans threads
 CONTEXTE : Protocoles custom, proxies, serveurs légers

CLIENT TCP :

```

reader, writer = await asyncio.open_connection('host', port)
reader, writer = await asyncio.open_connection('host', 8080,
    ssl=ssl_context, limit=2**16)
reader, writer = await asyncio.open_unix_connection('/tmp/socket.sock')

writer.write(b"données")
await writer.drain()         Flush le buffer
writer.writelines([b"a", b"b"])
await writer.drain()

données = await reader.read(n)      Lire jusqu'à n octets
données = await reader.readline()   Lire jusqu'à \n
données = await reader.readuntil(b"\r\n") Lire jusqu'au délimiteur
données = await reader.readexactly(n) Lire exactement n octets (IncompleteReadError)
reader.at_eof()                   True si EOF

writer.close()                  Fermer la connexion
await writer.wait_closed()       Attendre fermeture complète

```

```

writer.is_closing()                True si en cours de fermeture

# Infos
writer.get_extra_info('peername')  Adresse du pair
writer.get_extra_info('sockname')  Adresse locale
writer.get_extra_info('ssl_object') Objet SSL si TLS
writer.transport                   Transport sous-jacent

SERVEUR TCP :
async def handle_client(reader, writer):
    addr = writer.get_extra_info('peername')
    données = await reader.read(1024)
    writer.write(b"Réponse : " + données)
    await writer.drain()
    writer.close()
    await writer.wait_closed()

server = await asyncio.start_server(
    handle_client, '0.0.0.0', 8080,
    limit=2**16,                               Buffer max
    ssl=ssl_context,
)
server = await asyncio.start_unix_server(handle_client, path='/tmp/sock')

async with server:
    await server.serve_forever()                Tourner indéfiniment

server.sockets                        Liste des sockets
server.close()                       Arrêter d'accepter des connexions
await server.wait_closed()            Attendre fermeture de toutes les connexions

async with asyncio.start_server(handle_client, '0.0.0.0', 8080) as srv:
    await srv.serve_forever()

SOUS-PROCESSUS :
proc = await asyncio.create_subprocess_exec(
    'commande', 'arg1', 'arg2',
    stdout=asyncio.subprocess.PIPE,
    stderr=asyncio.subprocess.PIPE,
    stdin=asyncio.subprocess.PIPE,
)
proc = await asyncio.create_subprocess_shell(
    'ls -la | grep py',
    stdout=asyncio.subprocess.PIPE,
    stderr=asyncio.subprocess.PIPE,
)

stdout, stderr = await proc.communicate(input=b"données stdin")
await proc.wait()                      Attendre la fin
proc.returncode                       Code de retour (None si en cours)
proc.pid                             PID
proc.kill()                          Envoyer SIGKILL
proc.terminate()                     Envoyer SIGTERM
proc.send_signal(signal.SIGUSR1)      Envoyer un signal
proc.stdin.write(b"données")
await proc.stdin.drain()
données = await proc.stdout.read(1024)
ligne = await proc.stdout.readline()

DATAGRAMMES UDP :
# Client UDP
loop = asyncio.get_running_loop()
transport, protocol = await loop.create_datagram_endpoint(
    lambda: MonProtocoleUDP(),
    remote_addr=('127.0.0.1', 9999)
)
transport.sendto(b"données")
transport.close()

# Protocole UDP
class MonProtocoleUDP(asyncio.DatagramProtocol):
    def connection_made(self, transport):
        self.transport = transport
    def datagram_received(self, data, addr):
        print(f"Reçu de {addr}: {data}")

```

```
def error_received(self, exc):
    print(f"Erreur: {exc}")
def connection_lost(self, exc):
    pass
```

[io] Protocoles & Transports – API bas niveau

DESCRIPTION : Interface bas niveau pour protocoles réseau custom
 POURQUOI : Performance maximale, protocoles non-standard
 CONTEXTE : Implémentation de protocoles, wrappers SSL, proxies

PROTOCOLES :

```
class MonProtocole(asyncio.Protocol):
    def connection_made(self, transport):    Connexion établie
        self.transport = transport
    def data_received(self, data):           Données reçues
        self.transport.write(b"Réponse")
    def eof_received(self):                  EOF reçu (retourner True pour garder ouvert)
        return False
    def connection_lost(self, exc):          Connexion fermée
        pass
    def pause_writing(self):                 Buffer plein
        pass
    def resume_writing(self):                Buffer disponible
        pass

class MonProtocoleBuffer(asyncio.BufferedProtocol):
    def get_buffer(self, sizehint):          Fournir buffer pour la lecture
        return bytearray(sizehint)
    def buffer_updated(self, nbytes):        Buffer rempli de nbytes
        pass
```

TRANSPORTS :

```
transport.write(données)                    Écrire (non-bloquant)
transport.writelines([b"a", b"b"])
transport.write_eof()                       Envoyer EOF
transport.close()                           Fermer
transport.is_closing()                      True si en fermeture
transport.abort()                           Fermer sans flush
transport.can_write_eof()                   Supporte write_eof ?
transport.get_write_buffer_size()           Taille du buffer d'écriture
transport.get_extra_info('peername')        Adresse du pair
transport.get_extra_info('sockname')        Adresse locale
transport.set_write_buffer_limits(high, low) Seuils de remplissage buffer
transport.pause_reading()                   Suspendre la lecture
transport.resume_reading()                  Reprendre la lecture
```

CRÉER UNE CONNEXION :

```
loop = asyncio.get_running_loop()
transport, protocol = await loop.create_connection(
    MonProtocole, 'host', port)
transport, protocol = await loop.create_connection(
    MonProtocole, 'host', port, ssl=ssl_ctx)
transport, protocol = await loop.create_unix_connection(
    MonProtocole, '/tmp/sock')
server = await loop.create_server(
    MonProtocole, '0.0.0.0', port)
```

[io] Fichiers & Threads – asyncio + I/O bloquant

DESCRIPTION : Exécuter des opérations bloquantes sans bloquer la boucle
 POURQUOI : Les fichiers classiques sont bloquants – les déléguer à un thread
 CONTEXTE : Lecture de fichiers, appels de bibliothèques synchrones

THREAD POOL :

```
import asyncio
from concurrent.futures import ThreadPoolExecutor

# Méthode simple (Python 3.9+)
contenu = await asyncio.to_thread(open("f.txt").read)
résultat = await asyncio.to_thread(fn_bloquante, arg1, arg2)
```



```

résultat = await asyncio.to_thread(requests.get, "https://example.com")

# Méthode explicite
loop = asyncio.get_running_loop()
résultat = await loop.run_in_executor(None, fn_bloquante, arg)
résultat = await loop.run_in_executor(None, lambda: fn_bloquante(a, b))

# Pool de threads custom
with ThreadPoolExecutor(max_workers=10) as pool:
    résultat = await loop.run_in_executor(pool, fn_bloquante, arg)

PROCESS POOL :
from concurrent.futures import ProcessPoolExecutor

with ProcessPoolExecutor(max_workers=4) as pool:
    résultat = await loop.run_in_executor(pool, fn_cpu_intensive, données)

# Pour CPU-bound (calcul intensif)
resultats = await asyncio.gather(
    *[loop.run_in_executor(pool, fn_cpu, chunk) for chunk in chunks]
)

AIOFILES (fichiers async) :
# pip install aiofiles
import aiofiles

async with aiofiles.open("fichier.txt", mode='r') as f:
    contenu = await f.read()
async with aiofiles.open("fichier.txt", mode='r') as f:
    async for ligne in f:
        print(ligne)
async with aiofiles.open("sortie.txt", mode='w') as f:
    await f.write("contenu")
async with aiofiles.open("binaire.bin", mode='rb') as f:
    données = await f.read()

# Opérations sur le système de fichiers
await aiofiles.os.path.exists("fichier")
await aiofiles.os.rename("ancien", "nouveau")
await aiofiles.os.remove("fichier")
await aiofiles.os.makedirs("dossier", exist_ok=True)
await aiofiles.os.stat("fichier")

THREAD-SAFE (depuis autre thread) :
loop = asyncio.get_event_loop()

# Planifier une coroutine depuis un thread
future = asyncio.run_coroutine_threadsafe(ma_coro(), loop)
résultat = future.result(timeout=10)          Bloquant, attendre dans le thread

# Planifier un callback depuis un thread
loop.call_soon_threadsafe(callback, *args)

# asyncio.Queue est thread-safe pour la lecture seule
# Utiliser run_coroutine_threadsafe pour put/get depuis un thread

```

[coro] Générateurs Asynchrones – async for / async with

DESCRIPTION : Itérateurs et context managers asynchrones
POURQUOI : Streams de données, ressources à libérer proprement
CONTEXTE : Streams HTTP, curseurs BDD, connexions réseau

```

ASYNC ITERATOR :
class MonItérateur:
    def __aiter__(self):
        return self
    async def __anext__(self):
        valeur = await obtenir_suivant()
        if terminé:
            raise StopAsyncIteration
        return valeur

    async for item in MonItérateur():

```

```

    traiter(item)

GÉNÉRATEUR ASYNCHRONE :
    async def générer(n):
        for i in range(n):
            await asyncio.sleep(0)          Céder le contrôle
            yield i

    async for valeur in générer(10):
        print(valeur)

# Avec send / throw
    async def gen_avancé():
        valeur = yield "premier"          Reçoit la valeur envoyée
        yield f"second: {valeur}"

# Fermeture des générateurs async
    asyncio.run(loop.shutdown_asyncgens())    Fermer tous les générateurs async

ASYNC CONTEXT MANAGER :
    class MaRessource:
        async def __aenter__(self):          Entrée dans le bloc with
            await self.ouvrir()
            return self
        async def __aexit__(self, exc_type, exc, tb):    Sortie
            await self.fermer()
            return False                      False = ne pas supprimer l'exception

    async with MaRessource() as r:
        await r.utiliser()

# Avec contextlib
    from contextlib import asynccontextmanager

    @asynccontextmanager
    async def ma_ressource(param):
        ressource = await ouvrir(param)
        try:
            yield ressource
        finally:
            await ressource.fermer()

    async with ma_ressource("config") as r:
        await r.traiter()

ASYNC COMPREHENSIONS :
    résultats = [x async for x in générateur_async()]
    résultats = {x async for x in générateur_async()}
    résultats = {k: v async for k, v in paires_async()}
    résultats = [x async for x in générateur_async() if x > 0]

# await dans les compréhensions
    résultats = [await coro(x) for x in items]    ATTENTION : séquentiel
    résultats = await asyncio.gather(*[coro(x) for x in items])    Parallèle

```

[py] Gestion des erreurs & annulation

DESCRIPTION : Gérer les exceptions, timeouts et annulations proprement
POURQUOI : Robustesse, nettoyage des ressources, graceful shutdown
CONTEXTE : Services longue durée, pipelines critiques

EXCEPTIONS DANS LES TASKS :
 # Une exception dans une Task non awaited est silencieuse (warning seulement)
 task = asyncio.create_task(peut_échouer())
 # ... si jamais awaited, l'exception est perdue

 # Toujours await ou ajouter un handler
 task.add_done_callback(lambda t: t.exception() and print(t.exception()))

 # Récupérer l'exception
 try:
 résultat = await task
 except MaException as e:

```

    gérer(e)

EXCEPTION HANDLER DE LA BOUCLE :
def mon_handler(loop, context):
    print(f"Exception : {context['message']}")
    exc = context.get('exception')
    if exc:
        traceback.print_exception(type(exc), exc, exc.__traceback__)

loop.set_exception_handler(mon_handler)
loop.get_exception_handler()

# Champs de context :
# 'message'      : description de l'erreur
# 'exception'    : objet exception (si dispo)
# 'future'      : Future impliquée (si dispo)
# 'task'        : Task impliquée (si dispo)
# 'handle'      : Handle du callback
# 'source_traceback' : traceback de création

CANCELLATION PROPRE :
async def service():
    try:
        while True:
            await asyncio.sleep(1)
            await traitement()
    except asyncio.CancelledError:
        await nettoyage()          Nettoyage async possible
        raise                     Toujours re-raise !

# JAMAIS avaler CancelledError sans re-raise
# JAMAIS : except asyncio.CancelledError: pass

GRACEFUL SHUTDOWN :
async def main():
    server = await démarrer_serveur()
    try:
        await asyncio.Event().wait()          Tourner indéfiniment
    except asyncio.CancelledError:
        pass
    finally:
        await server.fermer()

# Avec signal
import signal

async def main():
    loop = asyncio.get_running_loop()
    stop = asyncio.Event()
    loop.add_signal_handler(signal.SIGTERM, stop.set)
    loop.add_signal_handler(signal.SIGINT, stop.set)
    await stop.wait()
    await nettoyage()

EXCEPTION GROUPS (Python 3.11+) :
try:
    async with asyncio.TaskGroup() as tg:
        tg.create_task(tâche1())
        tg.create_task(tâche2())
except* ValueError as eg:          Gère les ValueError du groupe
    for exc in eg.exceptions:
        print(f"ValueError : {exc}")
except* IOError as eg:
    for exc in eg.exceptions:
        print(f"IOError : {exc}")

# ExceptionGroup manuel
raise ExceptionGroup("plusieurs erreurs", [err1, err2, err3])

```

```

import asyncio, time

class RateLimiter:
    def __init__(self, appels_par_seconde):
        self.rate = appels_par_seconde
        self.tokens = appels_par_seconde
        self.updated_at = time.monotonic()

    async def acquérir(self, n=1):
        while True:
            now = time.monotonic()
            elapsed = now - self.updated_at
            self.tokens = min(self.rate, self.tokens + elapsed * self.rate)
            self.updated_at = now
            if self.tokens >= n:
                self.tokens -= n
                return
            await asyncio.sleep(n / self.rate - elapsed)

rl = RateLimiter(10)                                Max 10 appels/seconde
async def appel_limité():
    await rl.acquérir()
    return await ma_requête()

```

— CIRCUIT BREAKER

```

import asyncio, time

class CircuitBreaker:
    def __init__(self, seuil=5, délai_reset=60):
        self.échecs = 0
        self.seuil = seuil
        self.délai_reset = délai_reset
        self.dernier_échec = None
        self.ouvert = False

    async def appeler(self, coro):
        if self.ouvert:
            if time.monotonic() - self.dernier_échec > self.délai_reset:
                self.ouvert = False
                self.échecs = 0
            else:
                raise Exception("Circuit ouvert")
        try:
            résultat = await coro
            self.échecs = 0
            return résultat
        except Exception:
            self.échecs += 1
            self.dernier_échec = time.monotonic()
            if self.échecs >= self.seuil:
                self.ouvert = True
            raise

```

— RETRY AVEC BACKOFF

```

import asyncio, random

async def retry(coro_fn, *args, retries=3, délai=1.0, backoff=2.0,
                jitter=0.1, exceptions=(Exception,)):
    for tentative in range(retries):
        try:
            return await coro_fn(*args)
        except exceptions as e:
            if tentative == retries - 1:
                raise
            attente = délai * (backoff ** tentative)
            attente += random.uniform(0, jitter * attente)
            await asyncio.sleep(attente)

résultat = await retry(fetch, url, retries=5, délai=0.5)

```

— CACHE ASYNC

```

import asyncio
from functools import wraps

def cache_async(fn):

```

```

résultats = {}
en_cours = {}
@wraps(fn)
async def wrapper(*args):
    if args in résultats:
        return résultats[args]
    if args in en_cours:
        return await en_cours[args]
    task = asyncio.create_task(fn(*args))
    en_cours[args] = task
    try:
        résultats[args] = await task
        return résultats[args]
    finally:
        del en_cours[args]
return wrapper

```

```

@cache_async
async def fetch_utilisateur(id):
    return await bd.get(id)

```

— PIPELINE ASYNC

```

async def pipeline(*étapes, source):
    q_entrée = asyncio.Queue()
    q_sortie = q_entrée
    queues = [q_entrée]

    for étape in étapes:
        q = asyncio.Queue(maxsize=100)
        queues.append(q)
        asyncio.create_task(etape(q_sortie, q))
        q_sortie = q

    for item in source:
        await q_entrée.put(item)
    await q_entrée.put(None)  Sentinel

    résultats = []
    while True:
        item = await q_sortie.get()
        if item is None:
            break
        résultats.append(item)
    return résultats

```

— WORKER POOL

```

async def pool_workers(coro_fn, items, workers=10):
    q = asyncio.Queue()
    résultats = []

    async def worker():
        while True:
            item = await q.get()
            if item is None:
                break
            try:
                r = await coro_fn(item)
                résultats.append(r)
            except Exception as e:
                résultats.append(e)
            finally:
                q.task_done()

    tâches = [asyncio.create_task(worker()) for _ in range(workers)]
    for item in items:
        await q.put(item)
    for _ in range(workers):
        await q.put(None)  Sentinel par worker
    await q.join()
    for t in tâches:
        await t
    return résultats

```

— DEBOUNCE / THROTTLE

```

def debounce(délai):

```

```

def décorateur(fn):
    task = None
    @wraps(fn)
    async def wrapper(*args, **kwargs):
        nonlocal task
        if task:
            task.cancel()
        async def appeler():
            await asyncio.sleep(délai)
            await fn(*args, **kwargs)
        task = asyncio.create_task(appeler())
    return wrapper
return décorateur

```

```

@debounce(0.3)                                Attend 300ms après le dernier appel
async def traiter_input(texte):
    await rechercher(texte)

```

```

— BROADCAST —
class Broadcast:
    def __init__(self):
        self.abonnés = set()

    def abonner(self):
        q = asyncio.Queue()
        self.abonnés.add(q)
        return q

    def désabonner(self, q):
        self.abonnés.discard(q)

    async def publier(self, message):
        for q in self.abonnés:
            await q.put(message)

bus = Broadcast()
async def écouteur(nom):
    q = bus.abonner()
    async for msg in aiter_queue(q):
        print(f"{nom}: {msg}")

```

```
[py] Tests asynchrones – pytest-asyncio
```

```
INSTALL      : pip install pytest pytest-asyncio
```

```

import pytest
import asyncio

# pytest.ini ou pyproject.toml :
# [pytest]
# asyncio_mode = auto           Tous les tests async automatiquement

@pytest.mark.asyncio
async def test_ma_coro():
    résultat = await ma_coroutine()
    assert résultat == 42

# Fixture async
@pytest.fixture
async def client():
    c = await créer_client()
    yield c
    await c.fermer()

@pytest.mark.asyncio
async def test_avec_client(client):
    réponse = await client.get("/")
    assert réponse.status == 200

# Boucle partagée entre tests
@pytest.fixture(scope="session")
def event_loop():
    loop = asyncio.new_event_loop()

```

```

    yield loop
    loop.close()

# Timeout de test
@pytest.mark.asyncio
@pytest.mark.timeout(5)
async def test_avec_timeout():
    await asyncio.sleep(1)

# Mock async
from unittest.mock import AsyncMock, patch

async def test_avec_mock():
    with patch('module.fn_async', new_callable=AsyncMock) as mock:
        mock.return_value = 42
        résultat = await module.fn_async()
        assert résultat == 42
        mock.assert_awaited_once()

AsyncMock()                                Mock d'une coroutine
AsyncMock(return_value=42)
AsyncMock(side_effect=Exception("erreur"))
mock.assert_awaited()
mock.assert_awaited_once()
mock.assert_awaited_once_with(arg1, arg2)
mock.await_count                            Nombre d'appels
mock.await_args                            Derniers arguments

```

[py] Bibliothèques async tierces – Aperçu

HTTP CLIENT :

```

# aiohttp
import aiohttp
async with aiohttp.ClientSession() as session:
    async with session.get('https://example.com') as resp:
        texte = await resp.text()
        json = await resp.json()
        octets = await resp.read()
    async with session.post(url, json={'a': 1}) as resp:
        pass
aiohttp.ClientSession(connector=aiohttp.TCPConnector(limit=100))

# httpx async
import httpx
async with httpx.AsyncClient() as client:
    resp = await client.get('https://example.com')
    resp = await client.post(url, json={'a': 1})

```

HTTP SERVEUR :

```

# FastAPI (le plus populaire)
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
async def root():
    return {"message": "Bonjour"}
# uvicorn app:app --host 0.0.0.0 --port 8000

# aiohttp server
from aiohttp import web
async def handle(request):
    return web.Response(text="Bonjour")
app = web.Application()
app.add_routes([web.get('/', handle)])
web.run_app(app)

# Starlette
from starlette.applications import Starlette
from starlette.responses import JSONResponse
from starlette.routing import Route
async def homepage(request):
    return JSONResponse({'hello': 'world'})
app = Starlette(routes=[Route('/', homepage)])

```

BASE DE DONNÉES :

```
# asyncpg (PostgreSQL, très rapide)
import asyncpg
conn = await asyncpg.connect("postgresql://user:pass@localhost/db")
rows = await conn.fetch("SELECT * FROM t WHERE id = $1", 42)
row = await conn.fetchrow("SELECT * FROM t WHERE id = $1", 42)
val = await conn.fetchval("SELECT COUNT(*) FROM t")
await conn.execute("INSERT INTO t VALUES ($1, $2)", 1, 'texte')
await conn.executemany("INSERT INTO t VALUES ($1, $2)", [(1, 'a'), (2, 'b')])
async with conn.transaction():
    await conn.execute(...)
pool = await asyncpg.create_pool("postgresql://...", min_size=5, max_size=20)
async with pool.acquire() as conn:
    rows = await conn.fetch(...)

# aiosqlite
import aiosqlite
async with aiosqlite.connect("mabase.db") as db:
    async with db.execute("SELECT * FROM t") as cur:
        rows = await cur.fetchall()
    await db.execute("INSERT INTO t VALUES (?, ?)", (1, 'texte'))
    await db.commit()

# aiomysql
import aiomysql
conn = await aiomysql.connect(host='localhost', db='mabase',
                             user='user', password='pass')
cur = await conn.cursor(aiomysql.DictCursor)
await cur.execute("SELECT * FROM t WHERE id = %s", (42,))
rows = await cur.fetchall()
conn.close()
pool = await aiomysql.create_pool(host='localhost', db='mabase',
                                  user='user', password='pass',
                                  minsize=5, maxsize=20)

# SQLAlchemy async (Python 3.8+)
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker
engine = create_async_engine("postgresql+asyncpg://user:pass@localhost/db")
AsyncSessionLocal = sessionmaker(engine, class_=AsyncSession)
async with AsyncSessionLocal() as session:
    result = await session.execute(select(User))
    users = result.scalars().all()
```

CACHE :

```
# aioredis
import redis.asyncio as redis
r = await redis.from_url("redis://localhost")
r = redis.Redis(host='localhost', port=6379)
await r.set('clé', 'valeur')
await r.set('clé', 'valeur', ex=60)          TTL 60s
val = await r.get('clé')
await r.delete('clé')
await r.incr('compteur')
await r.lpush('liste', 'item')
items = await r.lrange('liste', 0, -1)
await r.hset('hash', 'champ', 'val')
await r.hgetall('hash')
pool = redis.ConnectionPool.from_url("redis://localhost", max_connections=20)
r = redis.Redis(connection_pool=pool)
```

MESSAGERIE :

```
# aio-pika (RabbitMQ)
import aio_pika
conn = await aio_pika.connect_robust("amqp://guest:guest@localhost/")
async with conn:
    channel = await conn.channel()
    exchange = await channel.declare_exchange('mon_exchange', aio_pika.ExchangeType.DIRECT)
    queue = await channel.declare_queue('ma_queue')
    await exchange.publish(aio_pika.Message(b"message"), routing_key='ma_queue')
    async for message in queue:
        async with message.process():
            print(message.body)
```



```
# aiokafka
from aiokafka import AIOKafkaProducer, AIOKafkaConsumer
producer = AIOKafkaProducer(bootstrap_servers='localhost:9092')
await producer.start()
await producer.send_and_wait('topic', b"message")
await producer.stop()
consumer = AIOKafkaConsumer('topic', bootstrap_servers='localhost:9092',
                             group_id="mon-groupe")

await consumer.start()
async for msg in consumer:
    print(msg.value)
```

WEBSOCKETS :

```
# websockets
import websockets
async with websockets.connect('ws://localhost:8765') as ws:
    await ws.send("Bonjour")
    réponse = await ws.recv()
async def handler(ws):
    async for message in ws:
        await ws.send(f"Echo: {message}")
async with websockets.serve(handler, "localhost", 8765):
    await asyncio.Future()
    Tourner indéfiniment
```

=====

RÉCAPITULATIF – ASYNCIO

=====

— QUAND UTILISER ASYNCIO —

CAS	OUTIL RECOMMANDÉ	RAISON
I/O réseau (HTTP, BDD, Redis)	asyncio + lib async	Une boucle, zéro thread
Serveur web haute perf.	FastAPI / aiohttp	ASGI, async natif
Scraping web	aiohttp + asyncio	N requêtes parallèles
CPU intensif	ProcessPoolExecutor	GIL n'aide pas asyncio
Code bloquant tiers	asyncio.to_thread()	Déporter dans thread
Fichiers locaux	aiofiles	Non-bloquant
BDD PostgreSQL async	asyncpg	Le plus rapide
BDD MySQL async	aiomysql	
Cache Redis async	redis.asyncio	

— GATHER VS TASKGROUP VS WAIT —

MÉTHODE	ANNULATION	SI ERREUR	RÉSULTATS	PYTHON
gather()	Oui (par défaut)		Liste ordonnée	3.4+
gather(return_exc.)	Non		Liste avec exc.	3.4+
TaskGroup	Oui (toutes)		t.result()	3.11+
wait(ALL_COMPLETED)	Non		Sets done/pending	3.4+
wait(FIRST_COMPLETED)	Non		Sets done/pending	3.4+
as_completed()	Non		Ordre de fin	3.4+

— ERREURS COMMUNES —

SYMPTÔME	CAUSE / SOLUTION
RuntimeWarning: coroutine never awaited	Oublié await devant coro()
boucle bloquée, latences élevées	Appel bloquant (time.sleep, requests...)
RuntimeError: no running event loop	asyncio.run() manquant
CancelledError avalée silencieusement	Ne pas faire except CancelledError: pass
Exception perdue dans une Task	Task jamais awaited – ajouter callback
Deadlock sur Lock	Lock acquis deux fois dans la même coro
ValueError: set_wakeup_fd only in main thread	Signaux depuis thread non-principal

— CHECKLIST PERFORMANCE —

- ✓ Utiliser uvloop (2-4x plus rapide que la boucle par défaut sur Linux/macOS)
- ✓ Limiter la concurrence avec Semaphore (éviter les flood de connexions)
- ✓ Pool de connexions BDD (asyncpg pool, aiomysql pool)
- ✓ asyncio.to_thread() pour tout code bloquant inévitable
- ✓ ProcessPoolExecutor pour CPU-bound (calcul, compression, chiffrement)
- ✓ gather() avec return_exceptions=True pour ne pas tout annuler sur une erreur
- ✓ TaskGroup pour une gestion d'erreur propre et structurée (Python 3.11+)
- ✓ Toujours re-raise CancelledError après nettoyage
- ✓ Utiliser asyncio.timeout() (Python 3.11+) plutôt que wait_for + task.cancel()
- ✗ Ne jamais appeler time.sleep() dans une coroutine → asyncio.sleep()
- ✗ Ne jamais utiliser requests → aiohttp ou httpx async

- x Ne jamais partager un objet non thread-safe entre boucles
- x Ne pas créer une nouvelle boucle dans chaque requête web

=====